

PISTY: Curriculum-Based Deep Reinforcement Learning of Ground and Aerial Striking in 1v1 Rocket League

Afonso Coelho^{1,2}, Diogo Amaral^{1,2}, Miguel Camocho Carvalho^{1,2},
Gonçalo Leão^{2,3}, Armando Sousa^{2,3}, and Luís Paulo Reis^{2,4}

¹FCUP - Faculty of Sciences, University of Porto, Portugal

²FEUP - Faculty of Engineering, University of Porto, Portugal

³INESC TEC - INESC Technology and Science, Portugal

⁴LIACC - Artificial Intelligence and Computer Science Laboratory, Portugal

Abstract—Autonomous control of agile vehicles in dynamic three-dimensional environments is a demanding problem for both classical controllers and standalone Deep Reinforcement Learning (DRL), due to continuous dynamics, complex physics, and extreme reward sparsity. We present PISTY, a car-like agent trained with DRL to perform ground and aerial striking maneuvers in a competitive 1v1 Rocket League simulation. To overcome the sparse feedback of terminal scoring events, we combine Proximal Policy Optimization (PPO) with a five-phase Curriculum Learning (CL) strategy, inspired by the game’s rank progression: Bronze (locomotion and ball control), Silver/Gold (ground scoring), Platinum (aerial interception), Diamond (competitive striking under near zero-sum rewards), and a Diamond+ refinement that reprioritizes final, goal-directed rewards. Training runs on GigaLearn, a high-throughput, natively parallel C++ framework, for roughly 4.2 billion environment timesteps, and is paired with self-play so the policy is refined against a non-stationary adversary. The resulting agent converges steadily across the five phases, learns to score consistently on the ground, and transfers to aerial striking while keeping optimization stable. The final policy also displays increasingly tactical use of bumps and demolitions, which are only lightly rewarded, to disrupt the opponent. These results support the combination of structured task decomposition and dense-to-sparse reward shaping as an effective route to continuous robotic control under adversarial constraints.

Index Terms—Action Masking, Autonomous Robots, Curriculum Learning, Deep Reinforcement Learning, Reward Shaping, Rocket League, Self-Play.

I. INTRODUCTION

Autonomous control of fast, agile robots in dynamic three-dimensional environments is a central open problem in robotics. Applications such as drone racing, legged locomotion, and autonomous driving share the same core difficulty: an agent must produce continuous, high-frequency control commands while reasoning about its own dynamics and other moving bodies, often with only sparse feedback about task success. Physics-based games are a popular testbed for these skills, exposing the same continuous-control and multi-agent challenges as real robotic platforms while remaining reproducible, safe, and inexpensive at scale.

We use Rocket League¹, a physics-driven car-and-ball game, as our testing ground: the agent drives a car-like body at high speed and, once airborne, must control its full three-dimensional orientation to strike a moving ball toward the opponent’s goal, coupling a hard locomotion problem with a competitive objective. This is a proxy for adversarial, continuous robotic control, and it is hard for both classical and learned methods: hand-engineered controllers must anticipate every situation and scale poorly, while applying Reinforcement Learning (RL) directly is hindered by a large continuous state space and acutely sparse rewards, since the natural objective (scoring) is seldom observed and the agent tends to stall in poor local optima.

To address the sparsity we adopt Curriculum Learning (CL), decomposing the task into increasingly difficult sub-tasks so that basic mechanics are acquired before harder maneuvers are demanded. Our aims are: (i) to design a five-phase curriculum, aligned with the game’s rank tiers, that gradually shifts the learning signal from dense mechanical shaping toward sparse, competitive, near zero-sum rewards; (ii) to reach competitive 1v1 play under this curriculum within a feasible single-workstation compute budget; (iii) to combine it with self-play so the policy is robust against a non-stationary adversary; and (iv) to characterize the resulting behavior with quantitative training metrics.

II. RELATED WORK

Deep Reinforcement Learning (DRL) [1] has been applied extensively to high-speed, continuous robotic control. Policy-gradient methods, and in particular Proximal Policy Optimization (PPO) [2], are widely used because they trade off sample efficiency and stability well in continuous domains; we adopt PPO as our base optimizer. However, learning complex behavior from scratch remains intractable when the target objective is highly sparse, which motivates additional structure on top of the base algorithm.

¹<https://www.rocketleague.com>

Curriculum Learning (CL), formalized by Bengio et al. [3], shapes the optimization landscape by presenting tasks in order of increasing difficulty. Narvekar et al. [4] survey CL specifically for RL and organize methods into task sequencing, reward shaping, and environment/state-setter modifications. Our curriculum draws on all three axes at once: at each phase boundary a scheduler rewrites the reward weights, the scenario (state-setter) distribution, and the PPO hyperparameters. Unlike surveyed methods that keep a single static shaping function, we schedule a dense-to-sparse transition explicitly tied to rank-inspired phases.

Competitive video games have been strong DRL benchmarks. AlphaStar (StarCraft II) [5] and OpenAI Five (Dota 2) [6] both reached top human level through self-play against past versions of the policy on sparse win/loss signals at enormous scale. We use the same self-play idea [7], so that emergent complexity arises from the arms race; our contribution lies in coupling this self-play with an explicit, hand-designed rank curriculum on a single-workstation budget, rather than in the sheer scale of data collection those systems relied on.

The systems most directly comparable to ours are RL agents for Rocket League built on RLGym. Necto and its successor Nexto² are self-play PPO agents trained with a largely flat, handcrafted reward at large distributed scale, reaching approximately Grand Champion level mainly through very large-scale data collection rather than structured task decomposition. Lucy-SKG [8] instead targeted sample efficiency, combining parameterizable reward-shape functions through a Kinesthetic Reward Combination scheme, and reported outperforming both. A recurring difficulty in such flat-reward settings is *mode collapse* [9], where the policy collapses onto a narrow, repetitive strategy (for example, always driving straight into the ball and never attempting aerials), together with catastrophic forgetting [10], [11] when the task shifts toward adversarial play. Our approach shares this environment and the PPO backbone but differs in method: a scheduler adapts the dense-to-sparse reward weighting and the PPO learning rate α and discount γ across explicit, rank-inspired phase boundaries (with the entropy coefficient β held fixed), and localized spatial reward filters restrict positive feedback to the attacking half of the pitch, which stabilizes aerial learning compared to unconstrained global shaping.

III. METHODOLOGICAL APPROACH

All the code was made publicly available in a Github repository³.

A. Simulation Environment and Interface

The environment is a 1v1 Rocket League match: two car-like agents and one ball on a closed arena with two goals. The objective of the game is to score by driving the ball into the opponent’s goal while preventing the opponent from doing the same, so a match rewards both offensive striking and

defensive positioning. A car can also *demolish* an opponent by colliding with it at high (supersonic) speed: the demolished car is destroyed and removed from play, respawning near its own goal after roughly 3 seconds. Demolitions are therefore a legal way to gain a brief positional and numerical advantage, which is why the agent is lightly rewarded for them. Fig. 1 shows a representative frame of the simulation used for training and evaluation.

The interface between the simulator and the learning algorithm is decisive for throughput. While *RLGym* v2⁴ offers a convenient Python implementation, we selected the *GigaLearn* framework⁵, a C++ engine that supports massively parallel rollouts with native PPO, built on the RocketSim physics engine⁶. By removing the overhead of high-level Python wrappers, it raises sample-generation throughput enough to make billions of timesteps feasible on a single workstation.

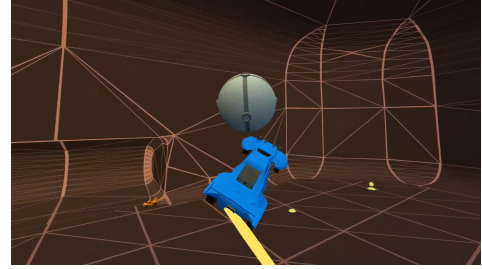


Figure 1: The 1v1 simulation environment used to train and evaluate PISTY.

B. Action Space and Masking

The policy is discrete and selects one of 90 actions, each mapping to the eight-element control vector

$$\mathbf{a} = (\textit{throttle}, \textit{steer}, \textit{pitch}, \textit{yaw}, \textit{roll}, \textit{jump}, \textit{boost}, \textit{handbrake}), \quad (1)$$

whose entries drive the car: *throttle* accelerates forward or backward, *steer* turns left or right on the ground, *pitch*, *yaw* and *roll* rotate the car about its three body axes while airborne, *jump* triggers a jump or flip, *boost* applies thrust, and *handbrake* enables a drifting turn. The first five entries take values in $\{-1, 0, 1\}$ and the last three in $\{0, 1\}$. Although the first five are physically continuous, each is discretized to three values, keeping the same action set Necto uses as a baseline; a discrete action space is also an optimization choice, since a categorical policy converges faster and more stably than a continuous one that must regress a distribution over every control dimension, which matters given the task’s reward sparsity.

The set is the union of a *ground* subset (wheel contact) and an *aerial* subset (airborne), contributing 24 and 66 actions after pruning redundant combinations (Table I), for a shared head of 90. At every step we apply *action masking*: only the ground

²<https://github.com/Rolv-Arild/Necto>

³https://github.com/Bugss05/PISTY_CL_DRL_in_Rocket_League

⁴<https://rlgym.org>

⁵<https://github.com/ZealanL/GigaLearnCPP>

⁶<https://github.com/ZealanL/RocketSim>

subset is enabled on the ground and only the aerial subset while airborne, boost actions only when boost is available, and jump actions only when a jump or flip is available or the car is upside down. Masked logits are set to $-\infty$ before the softmax, so the agent never samples an action the physics engine would ignore, and the mask selects the relevant subset per state.

Table I: Discrete Action Space Construction. The control-vector entries are t throttle, s steer, p pitch, y yaw, r roll, j jump, b boost, and h handbrake.

Subset	Vector form	Count
Ground	$(t, s, 0, s, 0, 0, b, h)$	24
Aerial	(b, y, p, y, r, j, b, h)	66
Total (shared head)		90

Ground: $t, s \in \{-1, 0, 1\}$, $b, h \in \{0, 1\}$, excluding $b=1$ with $t \neq 1$. Aerial: $p, y, r \in \{-1, 0, 1\}$, $j, b \in \{0, 1\}$, $h=1$ iff $j=1$ and $(p, y, r) \neq 0$, excluding $j=1$ with $y \neq 0$ and the neutral duplicate.

C. Observation Space

The agent perceives the environment through a normalized state vector $\mathbf{s} \in \mathbb{R}^{53}$, formed by concatenating four feature groups:

$$\mathbf{s} = [\mathbf{s}_{\text{ego}} \parallel \mathbf{s}_{\text{ball}} \parallel \mathbf{s}_{\text{opp}} \parallel \mathbf{s}_{\text{prev}}]^T, \quad (2)$$

where $\parallel$ denotes concatenation.

- **Ego vehicle ($\mathbf{s}_{\text{ego}}$, 18 features):** the car’s own physical state, namely its 3D position, its forward and up orientation vectors, its linear and angular velocities, and three boolean flags indicating whether the car is on the ground, whether a jump or flip is currently available, and whether the car is temporarily disabled after being demolished.
- **Ball ($\mathbf{s}_{\text{ball}}$, 9 features):** the ball’s 3D position, linear velocity and angular velocity.
- **Opponent ($\mathbf{s}_{\text{opp}}$, 18 features):** the same 18-feature description as the ego vehicle, for the single adversary. In the 1v1 setting there is one opponent ($N=1$), so this block appears exactly once; the same layout generalizes to N opponents by repetition.
- **Previous action ($\mathbf{s}_{\text{prev}}$, 8 features):** the eight-element control vector applied in the previous step, giving the policy short-term memory of its own last action. It is kept as a separate group because it describes the agent’s own command history, not any physical body in the scene.

The agent is given *infinite boost*: its boost tank never depletes, so it may accelerate or fly at any time. This removes boost-management from the task, letting the agent focus on locomotion and striking; features describing boost pads or amounts are therefore omitted.

D. Reward Design and Shaping

The reward mixes competitive, near zero-sum outcome signals with dense, non-zero-sum mechanical shaping terms. We describe what each family of terms measures; the per-phase weights are given later in Table V.

- 1) **Outcome and game events (near zero-sum):** reward for scoring (scaled by the ball’s entry speed and height), for pushing the ball toward the opponent’s goal, for shots on target, and for saves. These are the terms the opponent actively tries to prevent, so they are treated as zero-sum (Appendix A).
- 2) **Ball-contact quality:** reward for the instantaneous acceleration imparted to the ball on contact, for touching the ball at all, and for strong "power" touches above a target speed, with a small penalty for whiffs (swinging and missing).
- 3) **Approach and locomotion:** reward for moving toward the ball, for facing it, for driving fast, and a height-gated, time-decaying term that encourages controlled airborne time.
- 4) **Advanced aerial mechanics:** reward for touching the ball in the air, for flicks (flicking the ball off the car), for double-jump timing, for launching off walls, for wave-dashes (dash before hitting the ground), and for landing on all four wheels.
- 5) **Physical interaction (near zero-sum):** small rewards for bumping and for demolishing the opponent, present throughout training, which encourage the agent to use legal contact to disrupt the adversary.

E. Network Architecture and Training Configuration

The policy uses a Multi-Layer Perceptron (MLP) actor-critic with roughly 3 million trainable parameters. A shared feature-extraction trunk of three dense layers of 1024 units feeds two heads: a policy (actor) head and a value (critic) head, each of three dense layers of 256 units. The actor outputs 90 logits (masked as in Section III-B); the critic outputs a scalar state value. Fig. 2 shows the architecture.

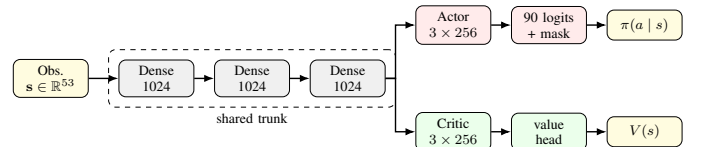


Figure 2: Actor-critic MLP: a shared three-layer trunk of 1024 units branches into a policy head (masked 90-way categorical) and a value head.

Optimization uses PPO. Table II lists the base hyperparameters; the learning rate α and discount γ are then overridden per phase, while the entropy coefficient β is kept fixed at 0.035 (Table VI).

F. Training Paradigm: Self-Play

The agent is trained entirely through *self-play*: it competes against historical checkpoints of its own evolving policy, so the environment is non-stationary and adversarial and the agent must surmount unpredictable opponents [7]. This is what lets the policy generalize to genuine 1v1 matches and acquire mechanics that a static opponent would never elicit. We do not train in a solo regime or against a fixed scripted bot; the

Table II: Base PPO Hyperparameter Configuration

Parameter	Base value
Learning rate ($\alpha_{policy} = \alpha_{critic}$)	2×10^{-4} *
Clip epsilon (ϵ)	0.1 – 0.2*
Timesteps per iteration	262 144
Batch size	262 144
Mini-batch size	131 072
Entropy coefficient (β)	0.035*
GAE discount (γ)	0.99*
Epochs per iteration	2
Parallel games	768
Rollout buffer	2048*
Optimizer	Adam

* Adapted per curriculum phase (Table VI).

only solo interaction occurs at *evaluation* time, after training completes at 4.2B timesteps, where we additionally measure performance against an empty net to isolate striking skill from defense (Section IV).

G. Curriculum Design

The observation and action spaces are fixed throughout training. What changes across phases are the initialization scenarios sampled by the state-setter, the reward weights, and the PPO hyperparameters. The total reward in phase k is a weighted sum:

$$R^{(k)} = \sum_i w_i^{(k)} r_i, \quad (3)$$

where r_i is a reward term and $w_i^{(k)}$ its phase weight. A scheduler advances to the next phase once the accumulated timesteps exceed that phase’s threshold, at which point it rewrites the reward and scenario weights and adjusts the hyperparameters, shifting learning from exploration toward refinement. Table V lists the reward weights per phase and Table VI the scenario weights and hyperparameters.

A *scenario* (or state-setter) defines how each training episode is initialized: it places the ball and both cars at chosen positions with chosen velocities and orientations, so that the agent repeatedly practices a specific game situation. Every scenario injects randomness into these initial positions, velocities and rotations of the ball and cars; this randomization does not change the skill being drilled, but keeps the policy from overfitting to a single fixed configuration and getting trapped in a local optimum. Besides the generic *kickoff* and *random* setups, we use four hand-designed scenarios:

- **Crossing:** the ball is launched high and in an arc from one of the wings into the attacking half, forcing the attacker to finish with an aerial while a goalkeeper defends near its own goal.
- **Passing:** a ground-level or chipped pass is played toward the attacker, who must receive and finish it against defenders positioned in front of their goal.
- **Shooting:** the ball rests on the ground in front of the goal and the attacker, placed just behind it, must strike it past a defender that harasses from the side.

- **Falling ball:** a ball drops almost vertically from high in the attacker’s own half, and the attacker, already in motion, must reach it and send it toward the opponent’s goal, which a keeper defends.

How often each scenario is drawn is set by its *scenario sampling weight* (Table VI). These weights are unnormalized coefficients: at sampling time they are normalized into probabilities, so a scenario of weight 3.0 is drawn three times as often as one of weight 1.0. Rewriting these weights at each phase boundary is what gradually exposes the agent to harder situations. The phases are:

1) Phase 1 (Bronze) – Moving and Controlling the Ball:

The first phase simply teaches the car to move and to keep the ball under control (basic driving and dribbling). Dense, local rewards dominate: strong touches and ball acceleration on contact, plus a reward for driving toward the ball. Goal-scoring rewards are switched off here so the agent is not distracted by a signal it can rarely obtain, and scenarios are split evenly between kickoffs and randomized positions.

2) Phase 2 (Silver/Gold) – Scoring and Ground Play:

Once contact is reliable, the agent is moved into a goal-oriented setting. The scoring reward is activated with a large weight, and pushing the ball toward the goal is prioritized over merely approaching it, while the generic contact-acceleration reward is disabled to force purposeful touches. Scenarios are skewed toward randomized match situations over kickoffs to diversify offensive trajectories.

3) **Phase 3 (Platinum) – Aerial Touches and Advanced Striking:** The third phase introduces aerial and wall-bound play. The scoring reward is raised and scaled by entry speed, aerial-touch and flick rewards are switched on, and dynamic scenarios dominate the state-setter so the agent frequently faces balls in the air and widens its aerial repertoire. Approach reward is reduced to enforce efficient positioning, and the planning horizon is extended (higher γ) to refine motor control. The entropy coefficient is unchanged; the measured policy entropy nonetheless declines as the policy sharpens.

4) **Phase 4 (Diamond) – Competitive Striking:** The fourth phase applies the full competitive task with minimal shaping, relying mostly on sparse, near zero-sum outcomes in lieu of dense feedback. The scoring reward reaches a large weight scaled by the ball’s entry speed, the state-setter emphasizes crosses and passes situations, and optimization is locked toward convergence with the lowest learning rate and the longest planning horizon.

5) **Phase 5 (Diamond+) – Outcome Refinement:** The fifth phase refines the Diamond reward from iteration 12 750 onward, shifting emphasis from technical execution toward final, goal-directed outcomes. The scoring reward is raised further and additionally scaled by the ball’s entry height, so decisive, well-placed finishes are favored. The technical contact terms (contact acceleration, power touch, and flick) are wrapped in the goal-direction filter of Appendix A so that they only pay out when the touch is aimed at goal, and aerial contact is rewarded through a goal-aim wrapper that scores the two-dimensional aim of the shot rather than

the touch alone. Consecutive-aerial-touch and air-proximity terms reward sustained, directed aerial control, and the wall-miss penalty is strengthened. The intent is to stop rewarding mechanics for their own sake and to reward them only insofar as they produce shots on goal.

IV. EXPERIMENTAL EVALUATION

All experiments ran on a single workstation: an AMD Ryzen 7 5800X3D CPU, 32 GB of DDR4-3600 RAM, and an AMD Radeon RX 9070 XT GPU with 16 GB of VRAM, under Ubuntu Linux with the ROCm compute stack. With the 262 144-timestep batch, GigaLearn sustains roughly 50 000 environment steps per second.

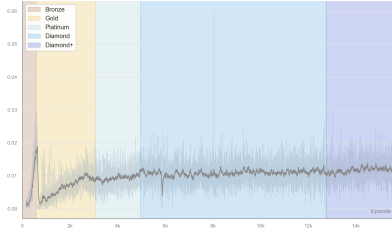
A. Convergence Across Phases

The per-metric behavior is discussed next; figures are grouped by what they measure, and dashed vertical lines mark the phase boundaries.

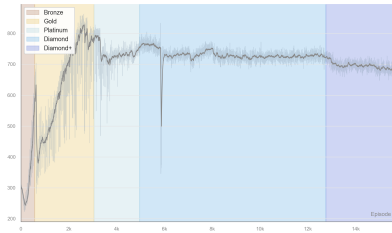
Table III: Cumulative Training Budget per Phase

Phase	Env. timesteps at onset
1 (Bronze)	0
2 (Silver/Gold)	~150M
3 (Platinum)	~800M
4 (Diamond)	~1.3B
5 (Diamond+)	~3.3B

Total training run: ~4.2B environment timesteps.



(a) Ball-contact reward.

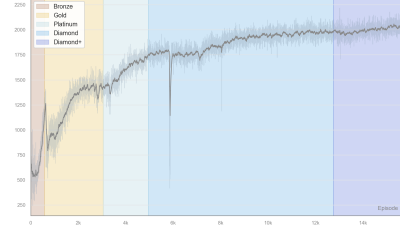


(b) Approach velocity toward the ball.

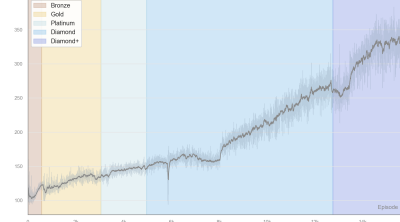
Figure 3: Phase-1 learning signals: dense contact and approach metrics over training iterations.

B. Skill-Rating Evaluation

The 1v1 skill rating in Fig. 6a is a relative strength score computed by an Elo system embedded in the self-play tracker: it measures the current policy against a pool of frozen past checkpoints of itself, so improvement is gauged against a

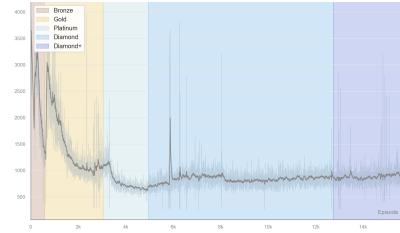


(a) Ball entry speed at the goal.

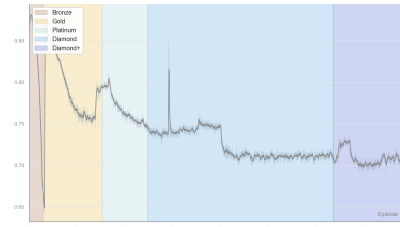


(b) Player's average ball touch height.

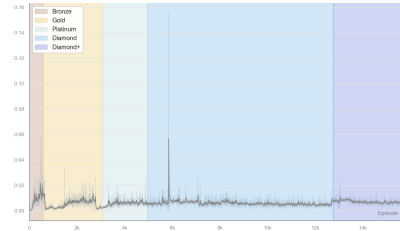
Figure 4: Offensive and aerial skill acquisition: scoring power and airborne contact over training iterations.



(a) Mean episode length.



(b) Policy entropy.



(c) Mean KL divergence.

Figure 5: Optimization health: episode length, policy entropy, and per-update KL divergence over training iterations.

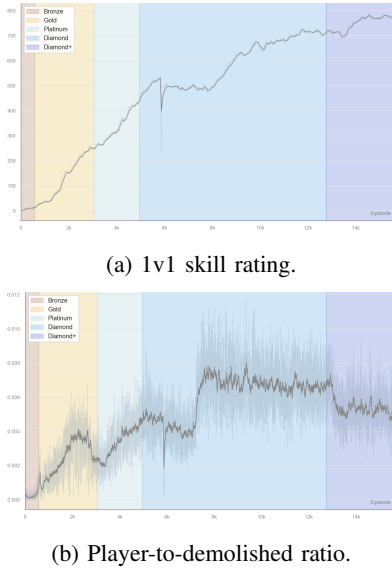


Figure 6: Outcome metrics: self-play skill rating and demolition ratio over training iterations.

moving, ever-stronger baseline rather than a fixed opponent. Every 16 iterations the current policy plays evaluation matches against a randomly selected past version across 8 parallel arenas, each ending when a goal is scored. The update is applied *per goal*: for a goal by a player of rating R_w against one of rating R_l , the scorer’s expected score is $E_w = 1/(1 + 10^{(R_l - R_w)/400})$, and the ratings move in zero-sum fashion, $R_w \leftarrow R_w + K(1 - E_w)$ and $R_l \leftarrow R_l - K(1 - E_w)$, with the standard divisor of 400 and a conservative gain $K = 5$. Ratings start at 0, are tracked separately per game mode, and each new snapshot inherits the current rating, so the scale does not reset and a rising curve reflects genuine, sustained gains in playing strength.

C. Discussion

The metrics coalesce into a coherent phase-by-phase story.

In **Phase 1**, dense contact feedback drives an immediate rise in the ball-contact reward (Fig. 3a) and a rapid increase in approach velocity (Fig. 3b). Mean episode length collapses from over 7 000 steps to under 1 500 (Fig. 5a) as the agent stops wandering and seeks the ball.

In **Phase 2**, activating the scoring reward pairs ball-tracking with directional offense: the ball’s entry speed at the goal climbs steadily (Fig. 4a) as the agent learns to strike harder and on target.

In **Phase 3**, aerial play is the hard part: the aerial-touch reward stays near zero at first and only grows after the agent solves the timing of leaving the ground (Fig. 4b). Throughout, the mean KL divergence stays bounded below 0.01 (Fig. 5c), showing that the decaying learning rate keeps policy updates small and stable even across the difficult handoff.

In **Phase 4**, the skill rating rises to its highest values (Fig. 6a) while the measured policy entropy settles to a low, stable floor (Fig. 5b), indicating a confident, converged policy.

A notable qualitative result is captured by the demolition ratio (Fig. 6b), which increases over training: the agent makes growing and increasingly well-timed use of bumps and demolitions to disrupt the opponent, revealing a proclivity for physical disruption. Although these interactions do carry a small, constant reward in every phase, their weight is minor relative to the dominant scoring and shaping terms (Table V), and no scenario or reward specifically teaches *when* or *whom* to demolish. The emergence of an effective, tactically-timed demolition behavior from such a small and undirected incentive is a surprising outcome, and suggests that self-play alone made physical disruption a useful sub-goal for winning.

In **Phase 5**, the Diamond+ revision sharpens finishing: gating the technical touch terms behind the goal-direction filter and adding a height-scaled goal bonus trades raw mechanical activity for shots that threaten the net. The change surfaces as a brief rise in policy entropy near iteration 13 000 (Fig. 5b) while the policy re-adapts, after which it settles again; this refined finisher is the policy evaluated in Table IV.

D. Scoring Performance

To answer whether the final (Diamond+) policy actually scores, we evaluated it from each scenario type, both against a defending opponent (*vs. opp.*) and against an empty net (no opponent), over the collated episode counts summarized in Table IV. In that table, t_{score} is the mean *time-to-score*: the average in-game time from episode start until a goal, computed only over the episodes that ended in a goal, $t_{\text{score}} = \frac{1}{|\mathcal{G}|} \sum_{e \in \mathcal{G}} t_e$, where $\mathcal{G}$ is the set of scoring episodes and t_e the time of the goal in episode e . It is a duration in seconds (lower is better), not a statistical t -value. Against an empty net the agent is highly reliable, converting 99% to 100% of episodes: it scores roughly every 5 s in shooting setups (≈ 12 goals per minute) and every 7 to 12 s in the harder crossing, passing, and falling-ball setups (≈ 5 to 8 goals per minute). Against a defending opponent, conversion falls to 65–86% and the mean time-to-score lengthens to 22–29 s in open-play scenarios, as expected, since the opponent both defends its goal and contests possession. The gap between the two conditions confirms that the offense learned under the curriculum transfers to full 1v1 matches, with the shooting scenario (closest to a direct strike) showing the smallest gap (85.8% vs. 99.0%).

Table IV: Scoring performance of the final policy by scenario, with and without a defending opponent: goal-conversion rate and mean time-to-score (s). Dashes mark scenarios without empty-net evaluation.

Scenario	Empty net		Vs. opp.	
	rate	t_{score}	rate	t_{score}
Kickoff	–	–	66.1%	28.6
Random	–	–	67.7%	26.0
Crossing	100.0%	7.2	71.9%	22.1
Passing	99.9%	10.3	66.8%	27.2
Shooting	99.0%	5.0	85.8%	9.5
Falling ball	99.9%	11.7	65.2%	28.7

Learned behavior per phase: Short videos showing what the agent has learned at the end of each of the five phases (Bronze, Silver/Gold, Platinum, Diamond, and Diamond+) are collected in a single public link.⁷

V. CONCLUSIONS AND FUTURE WORK

We presented PISTY, a DRL agent for 1v1 Rocket League trained with a five-phase, rank-inspired curriculum on top of PPO in a high-throughput C++ simulator. Our main contributions are: (i) a dense-to-sparse curriculum that schedules reward weights, scenario distributions, and PPO hyperparameters together across phases; (ii) an action-masked, shared ground/aerial discrete policy that respects the physics of the game; (iii) localized spatial reward filtering that stabilizes aerial learning; and (iv) an empirical characterization of convergence and emergent behavior across the five phases, reaching competitive 1v1 play within $\sim 4.2B$ environment timesteps.

The main results confirm the hypothesis: the curriculum agent learns reliable ball control (Phase 1), consistent ground scoring with rising shot power (Phase 2), aerial striking (Phase 3), a confident competitive policy with a monotonically rising self-play rating (Phase 4), and finally an outcome-refined finisher (Phase 5) that adheres to goal-directed shooting, converting almost every empty-net chance and staying effective against a defending opponent, all while keeping optimization stable (bounded KL, controlled entropy). The agent also learns to time bumps and demolitions to pressure the opponent.

Future work will extend the architecture to cooperative multi-agent play (2v2 and 3v3), which requires a team-spirit parameter in the zero-sum operator to balance individual and team reward. We also plan to remove the infinite-boost abstraction, reintroducing boost as a scarce resource, to study how resource management reshapes emergent rotation and aggression.

APPENDIX A

REWARD COMPOSITION: WRAPPERS

Individual reward terms are composed through *wrappers*, each of which encloses another reward term and transforms its output before it is added to the total. Two wrappers are relevant to this work.

a) Zero-sum wrapper: This wrapper turns a cooperative shaping signal into a competitive one, under the principle that what is good for the agent is bad for its opponent: if the agent scores, it gains reward and the opponent loses the same amount, so the policy learns that preventing the opponent is worthwhile, not only scoring. It is applied only to terms that it is advantageous for the opponent to prevent (scoring, ball-to-goal velocity, shots and saves, power touches, and speed). Mechanical shaping terms (airborne time, facing the ball, aerial control, wall play) are left unwrapped, because subtracting the opponent’s value there would only add noise,

since an opponent being airborne does not harm the agent. In the 1v1 setting the wrapper maps a child reward r_i to

$$R_i = r_i - \sigma r_i^{\text{opp}}, \quad (4)$$

the agent’s own term minus a scaled copy of the opponent’s same term. The opponent scale σ sets how much of the opponent’s reward is subtracted: $\sigma = 1$ is strictly zero-sum (the agent gains exactly what the opponent loses), whereas $\sigma < 1$ biases the agent toward aggression, since conceding costs less than scoring. We use $\sigma = 1$ for most zero-sum terms and a mildly aggressive $\sigma = 0.8$ for the goal bonus.

b) Goal-direction wrapper: This wrapper *filters* the enclosed reward by whether the touch actually sends the ball toward the opponent’s goal, and is applied to the contact and shooting terms (contact acceleration, power touch, and flick): hitting the ball hard is not enough, the contact must be aimed. It projects the ball’s straight-line trajectory onto the goal line and lets the reward pass only when the ball travels toward the target goal and between the posts (own-half touches can be left unfiltered by configuration). Crucially, a strong touch sent wide or in the wrong direction is not merely suppressed but *sign-inverted* (returning $-r_i$), so a powerful but misdirected shot is penalized rather than ignored, teaching the agent to shoot *on target* rather than simply hit the ball hard.

APPENDIX B

PER-PHASE CURRICULUM SCHEDULE

Tables V and VI give the full configuration rewritten by the scheduler at each phase boundary. Phase 1 uses the base hyperparameters of Table II; later phases override the learning rate, discount, and entropy as shown.

Table V: Reward Weights per Curriculum Phase. Diamond and Diamond+ share the same components; Diamond+ differs only in the goal reward ($70 \rightarrow 80$) and the wall-miss penalty ($-4 \rightarrow -5$), reflecting its shift toward final, goal-directed outcomes.

Reward Component	P1 Bronze	P2 Silv/Gold	P3 Plat	P4 Diam.	P5 Diam.+
Goal (speed/height scaled)	0.0	30.0	40.0	70.0	80.0
Ball-to-goal velocity	0.2	4.0	3.0	5.0	5.0
Touch acceleration [†]	1.0	0.0	1.5	1.5	1.5
Strong (power) touch [†]	10.0	2.0	0.85	1.0	1.0
Touch ball	40.0	3.0	0.05	0.00	0.00
Player-to-ball velocity	5.0	2.0	0.25	0.25	0.25
Face ball	0.5	0.2	0.3	0.15	0.15
Air (height-gated)	0.25	0.5	1.0	0.7	0.7
Speed	0.2	0.4	0.1	0.08	0.08
Whiff (penalty)	0.0	-0.1	-0.25	-0.25	-0.25
<i>Advanced mechanics & game events</i>					
Aerial touch (goal-aim)	0.0	0.0	10.0	7.0	7.0
Consec. aerial touch (goal-aim)	0.0	0.0	0.0	8.0	8.0
Air proximity	0.0	0.0	0.0	1.0	1.0
Double-jump boost	0.0	0.0	0.25	0.40	0.40
Flick [†]	0.0	0.0	4.5	12.0	12.0
Forward flip	0.0	0.0	3.5	0.8	0.8
Wall launch	0.0	0.0	0.2	0.2	0.2
Shot on target	0.0	0.0	2.5	6.5	6.5
Save	0.0	0.0	10.0	7.0	7.0
Wall-miss (penalty)	0.0	0.0	0.0	-4.0	-5.0
Bump	5.0	5.0	4.0	4.0	4.0
Demolition	3.0	3.0	3.0	3.0	3.0
Land on all fours	0.0	0.4	0.8	3.5	3.5
Wavedash	0.0	0.0	1.0	2.5	2.5

[†] In Diamond+, these technical touch terms are wrapped in the goal-direction filter (Appendix A), paying out only when the touch is aimed at goal.

⁷See videos at https://github.com/Bugss05/PISTY_CL_DRL_in_Rocket_League

Table VI: Scenario Weights and PPO Hyperparameters per Phase

	P1 Bronze	P2 S/Gold	P3 Plat	P4 Diam.	P5 Diam.+
<i>Scenario sampling weights</i>					
Kickoff	1.0	1.0	0.5	0.4	0.4
Random	1.0	1.0	1.0	1.0	1.0
Pass	0.0	0.5	0.4	1.0	1.0
Shooting	0.0	0.5	0.5	0.2	0.2
Falling ball	0.0	0.0	1.0	1.0	1.0
Cross	0.0	0.0	0.5	3.0	3.0
<i>PPO hyperparameters</i>					
Start (10^6 steps)	0	150	800	1300	3340
Learning rate α	2.0×10^{-4}	1.85×10^{-4}	1.5×10^{-4}	1.0×10^{-4}	1.0×10^{-4}
GAE γ	0.99	0.993	0.995	0.996	0.996
Entropy β	0.035	0.035	0.035	0.035	0.035

ACKNOWLEDGMENTS

We thank the RL Gym community, and especially ZealanL, for support in overcoming the challenges of this project.

This work is partially funded by national funds through FCT - Fundação para a Ciência e a Tecnologia, I.P., under the support UID/50014/2025 (<https://doi.org/10.54499/UID/50014/2025>). This work is partially financially supported by: Base Funding - UIDB/00027/2025 of the Artificial Intelligence and Computer Science Laboratory – LIACC - funded by national funds through the FCT/MCTES (PIDDAC).

REFERENCES

- [1] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, “Deep reinforcement learning: A brief survey,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*. ACM, 2009, pp. 41–48.
- [4] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, “Curriculum learning for reinforcement learning domains: A framework and survey,” *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.
- [5] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. Choi, R. Powell, T. Ewalds, P. Georgiev *et al.*, “Grandmaster level in starcraft ii using multi-agent reinforcement learning,” *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [6] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse *et al.*, “Dota 2 with large scale deep reinforcement learning,” *arXiv preprint arXiv:1912.06680*, 2019.
- [7] T. Bansal, J. Pachocki, S. Szymanski, I. Sutskever, and I. Mordatch, “Emergent complexity via multi-agent competition,” *arXiv preprint arXiv:1710.03748*, 2017.
- [8] V. Moschopoulos, P. Kyriakidis, A. Lazaridis, and I. P. Vlahavas, “Lucyskg: Learning to play rocket league efficiently using deep reinforcement learning,” *arXiv preprint arXiv:2305.15801*, 2023.
- [9] Y. Kossale, M. Airaj, and A. Darouichi, “Mode collapse in generative adversarial networks: An overview,” in *2022 8th International Conference on Optimization and Applications (ICOA)*, 2022, pp. 1–6.
- [10] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences (PNAS)*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [11] G. Leão, F. Almeida, E. Trigo, H. Ferreira, A. Sousa, and L. P. Reis, “Using deep reinforcement learning for navigation in simulated hallways,” in *2023 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*. IEEE, 2023, pp. 207–213.